Introduction to Python & Django

Web Development — Lecture 7

Seitbekov Sanzhar

KAZAKH-BRITISH TECHNICAL UNIVERSITY

KBTU · 2025 / 2026

Lecture Plan

PART 1 · 50 MIN

Python Fundamentals

→ Why Python for web?

→ Syntax, data types, variables

→ Sequence containers — list, tuple, dict, set

→ Functions & modules

→ Object-Oriented Programming & classes

PART 2 · 50 MIN

Django Framework

→ What is Django & why use it?

→ Project structure overview

→ settings.py — configuration

→ urls.py — routing

→ wsgi.py — deployment entry point

→ Live coding: first Django project

Why Python for Web Development?

Python in 2025

→ #1 most popular language (Stack Overflow 2024)

→ Readable, minimal boilerplate

→ Huge ecosystem — pip has 500k+ packages

→ Powers Instagram, Pinterest, Disqus

→ First-class AI/ML tooling (NumPy, PyTorch)

Mental model: Python lets you focus on *what to do*, not *how to tell the computer*.

Python web stack choices

Framework	Style	Use case
Django	Batteries-included	Full web apps
FastAPI	Modern, async	APIs, microservices
Flask	Micro-framework	Small apps, APIs

Today: We focus on Django — the most complete and widely used choice for full-stack Python web apps.

01

Python Syntax & Data Types

The building blocks — clean, readable, expressive

Python Syntax — The Basics

```python
# No semicolons, no curly braces
# Indentation IS the structure

# Variables — no type declaration
name = "Alice"
age  = 21
gpa  = 3.85
active = True

# Conditional
if gpa >= 3.5:
    print("Honours student")
elif gpa >= 2.0:
    print("Passing")
else:
    print("Academic warning")

# Loop
for i in range(5):
    print(i)  # 0 1 2 3 4
```

Built-in data types

Type	Example	Note
int	42	No size limit
float	3.14	64-bit double
str	"hello"	Immutable, Unicode
bool	True / False	Subclass of int
None	None	Like null / undefined

Key rule: Python uses *duck typing* — the type is determined at runtime, not declaration time. No `let`, `const`, or `var`.

02

Sequence Containers

list, tuple, dict, set — knowing when to use which

list & tuple

list — mutable, ordered

```python
students = ["Alice", "Bob", "Carol"]

# Indexing & slicing
students[0]    # "Alice"
students[-1]   # "Carol"
students[1:3]  # ["Bob", "Carol"]

# Mutating
students.append("Dave")
students.remove("Bob")
students.sort()

# List comprehension
upper = [s.upper() for s in students]

# Iterating
for s in students:
    print(s)
```

tuple — immutable, ordered

```python
# Tuples can't be changed after creation
point = (10, 20)
rgb   = (255, 128, 0)

# Unpacking
x, y = point
r, g, b = rgb

# Works as dict key (lists can't!)
grid = {(0, 0): "start", (3, 4): "end"}

# Named tuple for readability
from collections import namedtuple
Student = namedtuple('Student', ['name',
'gpa'])
alice = Student("Alice", 3.85)
alice.name  # "Alice"
```

Rule of thumb: Use `list` when the collection changes. Use `tuple` for fixed data — coordinates, RGB, database rows.

dict & set

dict — key-value store

PYTHON

```python
student = {
    "name": "Alice",
    "gpa":  3.85,
    "active": True
}

# Access
student["name"]              # "Alice"
student.get("grade", "N/A") # safe fallback

# Modify
student["gpa"] = 3.9
student["year"] = 2

# Iterate
for key, val in student.items():
    print(key, val)

# Dict comprehension
gpas = {s["name"]: s["gpa"] for s in
students}
```

set — unique, unordered

PYTHON

```python
tags = {"python", "web", "django"}

# No duplicates
tags.add("python")  # still 3 items

# Set operations — great for comparisons
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

a & b   # intersection → {3, 4}
a | b   # union        → {1,2,3,4,5,6}
a - b   # difference   → {1, 2}

# Fast membership test
"python" in tags  # True — O(1)
```

Quick guide: which container?

→ Ordered, mutable sequence → `list`

→ Ordered, fixed data → `tuple`

→ Key → value lookup → `dict`

→ Unique items, fast lookup → `set`

03

Functions & Modules

Writing reusable, composable Python code

Functions

```python
# Basic function
def greet(name):
    return f"Hello, {name}!"

# Default arguments
def create_student(name, gpa=0.0, active=True):
    return {"name": name, "gpa": gpa, "active": active}

# *args — variable positional args
def total(*numbers):
    return sum(numbers)

# **kwargs — variable keyword args
def build_profile(**kwargs):
    return kwargs

# Type hints (Python 3.5+)
def calculate_gpa(grades: list[float]) -> float:
    return sum(grades) / len(grades)

# Lambda — short anonymous function
square = lambda x: x ** 2
evens  = list(filter(lambda x: x % 2 == 0, range(10)))
```

Modules — organising code into files

```python
# utils.py
def format_gpa(gpa: float) -> str:
    return f"{gpa:.2f}"

MAX_STUDENTS = 30

# ─── main.py ───
# Import the whole module
import utils

# Import specific names
from utils import format_gpa, MAX_STUDENTS

# Alias
import utils as u

# Standard library examples
import os
import json
import datetime
from pathlib import Path
```

Convention: Every `.py` file is a module. A folder with `__init__.py` is a *package*. Django projects are built from packages.

04

Object-Oriented Programming

Classes, instances, inheritance — modelling the real world

Classes & Instances

```python
class Student:
    # Class variable — shared by all instances
    university = "KBTU"

    # Constructor
    def __init__(self, name: str, gpa: float):
        # Instance variables
        self.name = name
        self.gpa  = gpa

    # Instance method
    def is_honours(self) -> bool:
        return self.gpa >= 3.5

    # String representation
    def __str__(self) -> str:
        return f"{self.name} (GPA: {self.gpa})"

    # Class method — alternative constructor
    @classmethod
    def from_dict(cls, data: dict):
        return cls(data["name"], data["gpa"])
```

Using the class

```python
# Create instances
alice = Student("Alice", 3.85)
bob   = Student("Bob",   2.40)

print(alice)            # Alice (GPA:
3.85)
print(alice.is_honours()) # True
print(Student.university) # KBTU

# From dict
data = {"name": "Carol", "gpa": 3.7}
carol = Student.from_dict(data)
```

Inheritance

```python
class GradStudent(Student):
    def __init__(self, name, gpa, thesis):
        super().__init__(name, gpa)
        self.thesis = thesis

    def __str__(self):
        return f"{super().__str__()}
[{self.thesis}]"
```

05

Django Framework Intro

The web framework for perfectionists with deadlines

What Is Django?

Django is "batteries included"

→ ORM — talk to a database using Python classes

→ Admin panel — auto-generated CRUD interface

→ Authentication — users, sessions, permissions

→ URL routing — map URLs to Python functions

→ Templating engine — generate HTML server-side

→ Form handling & validation

→ Security — CSRF, XSS, SQL injection protection

Django follows MTV

Layer	Responsibility
Model	Data & database logic
Template	HTML presentation layer
View	Business logic, glues M & T

> **vs MVC:** Django's "View" is closer to a Controller. The Template is the View. Same concept, different naming.

Install & create project

SHELL

```shell
# Install Django
pip install django

# Create a project
django-admin startproject mysite

# Run development server
cd mysite
python manage.py runserver
```

06

Project Structure

settings.py · urls.py · wsgi.py

Django Project Structure

```shell
mysite/
├── manage.py          # CLI entry point
├── mysite/            # Project package
│   ├── __init__.py
│   ├── settings.py    # All configuration
│   ├── urls.py        # Root URL router
│   ├── asgi.py        # Async server entry
│   └── wsgi.py        # WSGI server entry
└── students/          # Your first app
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── models.py      # Database models
    ├── views.py       # Request handlers
    ├── urls.py        # App-level routes
    ├── tests.py
    └── migrations/    # DB schema history
```

Project vs App: A Django *project* is the entire website. An *app* is one self-contained feature (students, blog, payments…). One project = many apps.

Creating an app

```shell
# Create a new app inside your project
python manage.py startapp students
```

manage.py — your command center

Command	Does
`runserver`	Start dev server
`startapp`	Create new app
`makemigrations`	Generate DB migrations
`migrate`	Apply migrations
`createsuperuser`	Create admin account
`shell`	Interactive Python shell

settings.py — Project Configuration

PYTHON

```python
# settings.py
from pathlib import Path

# Build paths inside the project
BASE_DIR = Path(__file__).resolve().parent.parent

# SECURITY — keep this secret in production!
SECRET_KEY = 'django-insecure-...'

# Debug mode — ALWAYS False in production
DEBUG = True

ALLOWED_HOSTS = ['localhost', '127.0.0.1']

# Apps registered in this project
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'students',   # ← your app here
]
```

PYTHON

```python
# Database — SQLite by default (dev only)
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

# For PostgreSQL in production:
# 'ENGINE': 'django.db.backends.postgresql'
# 'NAME': 'mydb', 'USER': 'postgres', ...

# Static files (CSS, JS, images)
STATIC_URL = 'static/'

# Default primary key type
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

# Templates
TEMPLATES = [{
    'BACKEND':
'django.template.backends.django.DjangoTemplates',
    'DIRS': [BASE_DIR / 'templates'],
    'APP_DIRS': True,
}]
```

Never commit SECRET_KEY to Git. Use environment variables (`python-decouple` or `os.environ`) in real projects.

urls.py — Routing

Root urls.py — the project router

PYTHON

```python
# mysite/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    # Built-in admin panel
    path('admin/', admin.site.urls),

    # Delegate /students/* to the app
    path('students/', include('students.urls')),

    # Root URL → home view
    path('', include('students.urls')),
]
```

App-level urls.py

PYTHON

```python
# students/urls.py
from django.urls import path
from . import views

app_name = 'students'   # namespace

urlpatterns = [
    path('',           views.list_view,
name='list'),
    path('<int:pk>/', views.detail_view,
name='detail'),
    path('new/',       views.create_view,
name='create'),
]
```

URL patterns explained

Pattern	Matches
`''`	Exact empty — root
`'students/'`	Literal string
`'<int:pk>/'`	Integer captured as pk
`'<str:slug>/'`	String captured as slug
`'<uuid:id>/'`	UUID format

> **Naming URLs** with name= lets you use `reverse('students:detail', args=[pk])` in views and `{% url 'students:list' %}` in templates — no hardcoded paths.

A minimal View

PYTHON

```python
# students/views.py
from django.http import HttpResponse

def list_view(request):
    return HttpResponse("Hello from Django!")
```

wsgi.py — Deployment Entry Point

What is WSGI?

Web Server Gateway Interface — the standard interface between Python web apps and production servers (Gunicorn, uWSGI).

PYTHON

```python
# mysite/wsgi.py
import os
from django.core.wsgi import
get_wsgi_application

# Point to your settings module
os.environ.setdefault(
    'DJANGO_SETTINGS_MODULE',
    'mysite.settings'
)

# The application callable — the production
server
# calls this to handle every HTTP request
application = get_wsgi_application()
```

Request lifecycle in production

- **Browser** — Sends HTTP request to the server
- **Nginx** — Reverse proxy — handles static files, SSL
- **Gunicorn** — WSGI server — runs worker processes
- **wsgi.py** — Django entry — `application(environ, start_response)`
- **View** — Your Python code runs, returns response

WSGI vs ASGI

	WSGI	ASGI
File	`wsgi.py`	`asgi.py`
Style	Synchronous	Async / sync
Server	Gunicorn	Uvicorn / Daphne
Use	Standard Django	WebSockets, async views

Putting It All Together

Request → Response flow

1. Browser requests `/students/42/`
2. `urls.py` matches `<int:pk>`, calls `detail_view(request, pk=42)`
3. `views.py` queries the `Student` model via ORM
4. View renders a `template` with the student data
5. Django returns an `HttpResponse` to the browser

What we build today (live coding)

- → Create a Django project from scratch
- → Create a `students` app
- → Register it in `INSTALLED_APPS`
- → Add a `Student` model with name, gpa, active
- → Wire up URLs — list and detail
- → Write two simple views returning HTML
- → Register model in Django admin panel
- → Run `makemigrations` & `migrate`

Live coding goal: By end of lecture you have a running Django project with a working admin panel and two URL endpoints.

Key Takeaways

🐍

Python

→ Dynamic typing, indentation-based

→ `list`, `tuple`, `dict`, `set`

→ Functions with defaults, *args, **kwargs

→ Classes with `__init__`, inheritance

→ Every file = a module

🌐

Django

→ MTV pattern — Model / Template / View

→ One project → many apps

→ Batteries included: ORM, admin, auth

→ `manage.py` is your command center

→ Apps must be in `INSTALLED_APPS`

⚙️

Config Files

→ `settings.py` — all configuration

→ `urls.py` — routing table

→ `wsgi.py` — production entry point

→ Named URLs → use `reverse()`

→ Never hardcode secret keys

Next lecture: Django Models & ORM — defining your database schema with Python classes, migrations, and querying data.

Q & A

Ask anything — we'll debug it together.

How to reach me

👤 Seitbekov Sanzhar

Lecturer

✉ s.seitbekov@kbtu.kz

📍 Office 178

👥 Course chat

Microsoft Teams

Team code: **ksg13tw**

→ Announcements + materials

→ Homework questions

→ Useful links (cheatsheets, docs)